Number Theory in Competitive Programming

KCPC Training

Overview

Attendance

Our Socials

Instagram

Discord

WhatsApp

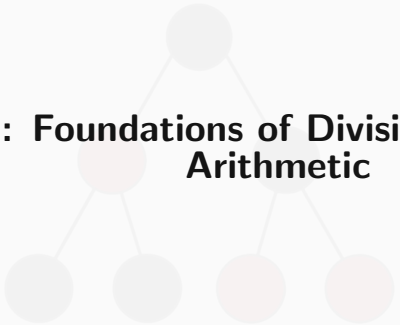
Core Ideas

Want to be able to deal with systems of factorisations, division and remainders concretely and easily.

Playing around with factorisations or sequences of numbers as a kid may have revealed patterns or similarities that we want to now formalise.

Convention: we take the naturals $\mathbb{N}$ to be the integers ≥ 1

Module 1: Foundations of Divisibility and Modular Arithmetic



Division Algorithm

Division Algorithm Given two integers, we can always divide them (given they're not 0) and end up with a ratio/**quotient** and a **remainder**.

$$A, B \in \mathbb{Z}, B \neq 0, \exists q, r \in \mathbb{Z}, r \in [0, B-1] : A = qB + r$$

Basics

We claim a number is a **factor** of another when the remainder is 0.

We say A divides B; A is a factor of B; B is a **multiple** of A and use the notation $A|B$ if there exists an integer C such that $B = A * C$

$\pm 1, \pm B$ are both factors of B also, and all others are called **proper factors**.

A natural with no proper factors is a **prime**.

Greatest Common Divisor (GCD)

Definition: The largest positive integer that divides both A and B without leaving a remainder.

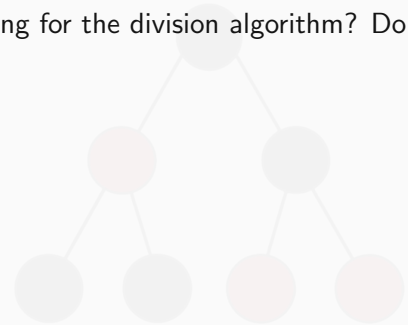
We call it G, and define it to be the natural such that if G is a common factor of A, B so that $G|A, G|B$ then for all other common factors F, $F|G$

The Naïve Approach:

- → Iterate from $\min(A, B)$ down to 1.
- → **Complexity:** $\mathcal{O}(\min(A, B))$.
- → *Verdict:* Too slow for CP bounds (where $A, B \leq 10^{18}$).

Extending the Division Algorithm

What if we kept going for the division algorithm? Do we gain any valuable insight?



Extending the Division Algorithm

What if we kept going for the division algorithm? Do we gain any valuable insight? Set it up as:

$$
\begin{aligned}
A &= q_1 B + r_1 \\
B &= q_2 r_1 + r_2 \\
r_1 &= q_3 r_2 + r_3 \\
&\vdots \\
r_{n-2} &= q_n r_{n-1}
\end{aligned}
$$

Extending the Division Algorithm

What if we kept going for the division algorithm? Do we gain any valuable insight? Set it up as:

$$\begin{aligned} A &= q_1 B + r_1 \\ B &= q_2 r_1 + r_2 \\ r_1 &= q_3 r_2 + r_3 \\ &\vdots \\ r_{n-2} &= q_n r_{n-1} \end{aligned}$$

Claim: then the highest common factor is r_{n-1}.

Interlude: Introducing Modular Arithmetic

Motivation: Is there a fundamental way to quickly express factorisations and remainders? Wouldn't it be nice if it also helped else condense theorems about factorisations and primality?

Interlude: Introducing Modular Arithmetic

Motivation: Is there a fundamental way to quickly express factorisations and remainders? Wouldn't it be nice if it also helped else condense theorems about factorisations and primality?

We introduce **modular arithmetic**. Consider $A = qB + r$ we can express this in the form $A(\mathrm{mod}\, B) \equiv r$.

A 'give or take' some B's is r

What is $4(\mathrm{mod}2), 19(\mathrm{mod}7), -8(\mod 13)$?

The Euclidean Algorithm

Core Intuition: If $A > B$, then any divisor of A and B also divides their difference, $A - B$. By extension, it divides the remainder of $A \pmod{B}$.

The Recurrence:

$$\gcd(A, B) = \gcd(B, A \bmod B)$$

Base Case:

$$\gcd(A, 0) = A$$

Euclidean Algorithm: Implementation

Pseudocode (Recursive):

```
function gcd(A, B):
    if B == 0:
        return A
    return gcd(B, A mod B)
```

Complexity Analysis:

- → **Time:** $\mathcal{O}(\log(\min(A, B)))$
- → **Space:** $\mathcal{O}(\log(\min(A, B)))$ due to call stack.
- → *Note:* The worst-case input for GCD is consecutive Fibonacci numbers.

Extended Euclidean Algorithm: The "Why"

Sometimes we need more than just the GCD. We need to express the GCD as a linear combination of A and B.

Bézout's Identity: For any integers A and B, there exist integers x and y such that:

$$A \cdot x + B \cdot y = \gcd(A, B)$$

Why in CP?

- → Solving Linear Diophantine Equations.
- → Computing Modular Inverses.

Extended Euclidean Algorithm: Changing Directions

Remember how we find the greatest common divisor:

$$\begin{aligned} A &= q_1 B + r_1 \\ B &= q_2 r_1 + r_2 \\ r_1 &= q_3 r_2 + r_3 \\ &\vdots \\ r_{n-3} &= q_{n-1} r_{n-2} + r_{n-1} \\ r_{n-2} &= q_n r_{n-1} = q_n G \end{aligned}$$

Now instead try working backwards: can express things like:

$$G = \frac{1}{q_n q_{n-1}} (r_{n-3} - r_{n-1})$$

Extended Euclidean Algorithm: The "How"

Assume we already have the coefficients x_1, y_1 for the next recursive step:

$$B \cdot x_1 + (A \bmod B) \cdot y_1 = \gcd(A, B)$$

We know that $A \bmod B = A - \left\lfloor \frac{A}{B} \right\rfloor \cdot B$. Substitute this into the equation:

$$B \cdot x_1 + \left(A - \left\lfloor \frac{A}{B} \right\rfloor \cdot B \right) \cdot y_1 = \gcd(A, B)$$

Rearranging terms to isolate A and B:

$$A \cdot y_1 + B \cdot \left(x_1 - \left\lfloor \frac{A}{B} \right\rfloor \cdot y_1 \right) = \gcd(A, B)$$

Extended Euclidean Algorithm: Implementation

Matching coefficients gives us our transition: $x = y_1$ and $y = x_1 - \left\lfloor \frac{A}{B} \right\rfloor \cdot y_1$.

Pseudocode:

```
function extended_gcd(A, B):
    if B == 0:
        return (A, 1, 0)  // gcd, x, y
    g, x1, y1 = extended_gcd(B, A mod B)
    x = y1
    y = x1 - floor(A / B) * y1
    return (g, x, y)
```

Time Complexity: $\mathcal{O}(\log(\min(A, B)))$

Modular Arithmetic: Division

Addition, subtraction, and multiplication distribute nicely over modulo M. **Division does not.**

$$(A/B) \bmod M \neq ((A \bmod M)/(B \bmod M)) \bmod M$$

Instead of dividing by B, we must multiply by its **Modular Inverse**, denoted as B^{-1}.

Definition of Modular Inverse:

$$B \cdot B^{-1} \equiv 1 \pmod{M}$$

Modular Inverses: Existence

Crucial Rule: The modular inverse $B^{-1} \pmod{M}$ exists **if and only if** B and M are coprime (i.e., $\gcd(B, M) = 1$).

If $\gcd(B, M) > 1$, it is mathematically impossible to divide by B under modulo M.

Recall: for M a prime p, every integer smaller than p is coprime! Working modulo p is extremely convenient!

Method 1: Fermat's Little Theorem

Theorem: If M is prime, and A is not divisible by M:

$$A^{M-1} \equiv 1 \pmod{M}$$

Multiply both sides by A^{-1}:

$$A^{M-2} \equiv A^{-1} \pmod{M}$$

Implementation: Calculate $A^{M-2} \pmod{M}$ using Binary Exponentiation.

→ **Requirement:** M must be prime.

→ **Time Complexity:** $\mathcal{O}(\log M)$

Method 2: Extended Euclidean

What if M is not prime? As long as $\gcd(A, M) = 1$, we use Bézout's Identity:

$$A \cdot x + M \cdot y = 1$$

Take modulo M on both sides. The $M \cdot y$ term vanishes:

$$A \cdot x \equiv 1 \pmod{M}$$

Therefore, x (computed via `extended_gcd`) is the modular inverse of A modulo M.

→ **Requirement:** $\gcd(A, M) = 1$.

→ **Time Complexity:** $\mathcal{O}(\log(\min(A, M)))$.

Pro-Tips & CP "Gotchas"

- → **Negative Modulo in C++/Java:** The % operator can return negative values. Always sanitize!
  `safe_mod = (x % M + M) % M`
- → **Extended GCD Normalization:** The x returned by `extended_gcd` might be negative. Apply the safe modulo technique above before using it as an inverse.
- → **Overflows:** When multiplying large numbers before taking the modulo, use 64-bit integers (`long long` in C++).

Module 2: Exponentiation and Sequence Optimization

Binary Exponentiation: The Objective

The Problem: Calculate $A^B \pmod{M}$ efficiently.

The Naïve Approach:

- → Multiply A by itself B times.
- → **Complexity:** $\mathcal{O}(B)$ time.
- → *Verdict:* Time Limit Exceeded (TLE) when B is large (e.g., $B \geq 10^9$).

Binary Exponentiation: The Intuition

Core Idea: Divide and Conquer. We can cut the work in half at each step by using the properties of exponents.

The Recurrence:

- → **If B is even:** $A^B = \left(A^{B/2}\right)^2$
- → **If B is odd:** $A^B = A \cdot \left(A^{(B-1)/2}\right)^2$
- → **Base Case:** $A^0 = 1$

Example (3^{13}): Instead of 13 multiplications, we compute $3 \to 3^2 \to 3^3 \to 3^6 \to 3^{12} \to 3^{13}$ requiring only 5 multiplications.

Binary Exponentiation: Implementation

Pseudocode (Iterative):

```
function binpow(A, B, M):
    A = A mod M
    result = 1
    while B > 0:
        if B is odd:     // (B & 1) == 1
            result = (result * A) mod M
        A = (A * A) mod M
        B = floor(B / 2) // B >>= 1
    return result
```

Complexity: Time $\mathcal{O}(\log B)$, Space $\mathcal{O}(1)$.

Fibonacci Sequence: Hitting the Wall

The Sequence: $F_n = F_{n-1} + F_{n-2}$, with $F_0 = 0, F_1 = 1$.

Standard Dynamic Programming:

- → Iterate from 2 to N, storing previous values.
- → **Complexity:** $\mathcal{O}(N)$ time.
- → **The Constraint Wall:** In many CP problems, N can be up to 10^{18}. An $\mathcal{O}(N)$ algorithm will fail. We need a logarithmic approach.

Matrix Exponentiation: Mathematical Model

We can represent linear recurrences as matrix multiplications.

To transition from the state $\begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}$ to $\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$, we multiply by a transition matrix:

$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n-1} \\ F_{n-2} \end{pmatrix}$$

Matrix Exponentiation: The $\mathcal{O}(\log N)$ Leap

By expanding the recurrence, we reach the base state:

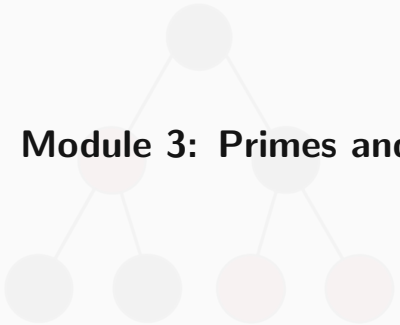
$$\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

The breakthrough: Matrix multiplication is associative. This means we can compute the matrix power $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1}$ using **Binary Exponentiation**!

Complexity: Matrix multiplication takes $\mathcal{O}(K^3)$ where $K = 2$. Doing this $\log N$ times yields a total time complexity of $\mathcal{O}(2^3 \log N) = \mathcal{O}(\log N)$.

Pro-Tips & CP "Gotchas"

- → **Identity Matrix:** When implementing matrix binary exponentiation, always initialize your result matrix to the Identity Matrix (diagonal of 1s, rest 0s). It acts as the integer "1" for matrices.
- → **Modulo at Every Step:** In both Matrix Exponentiation and Fast Doubling, apply the modulo operator after *every single* addition and multiplication to prevent integer overflow.
- → **Negative Modulo in Fast Doubling:** The formula $F_{2k} = F_k \cdot (2F_{k+1} - F_k)$ involves subtraction. Ensure you use the safe modulo technique: `F_2k = (F_2k % M + M) % M`.



Module 3: Primes and Sieves

Generating Primes: The Need for Sieves

The Scenario: You need to find all prime numbers up to N, or rapidly answer queries about whether $K \leq N$ is prime.

Naïve Approach:

- → Iterate i from 2 to N.
- → Run trial division up to $\sqrt{i}$ for each number.
- → **Complexity:** $\mathcal{O}(N\sqrt{N})$.
- → *Verdict:* Too slow for $N = 10^7$. We need to precompute primes efficiently.

The Sieve of Eratosthenes: Intuition

Core Idea: Instead of checking if a number has divisors, we let prime numbers "announce" their multiples as composite.

The Process:

1. Assume all numbers from 2 to N are prime.
2. Find the first uncrossed number (which must be prime).
3. Cross out all of its multiples.
4. Repeat until you have processed up to $\sqrt{N}$.

The Sieve of Eratosthenes: Implementation

Optimization: We only need to start crossing out multiples from $i \cdot i$, because any smaller multiple $i \cdot k$ (where $k < i$) would have already been crossed out by the smaller prime factor k.

```
function sieve_eratosthenes(N):
    is_prime = boolean array of size N+1, initialized to true
    is_prime[0] = is_prime[1] = false

    for i from 2 to sqrt(N):
        if is_prime[i]:
            for j from i * i to N step i:
                is_prime[j] = false

    return all i where is_prime[i] is true
```

Complexity of the Standard Sieve

Time Complexity Analysis: The inner loop runs N/p times for each prime p. The total number of operations is proportional to:

$$N \cdot \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots \right)$$

The sum of the reciprocals of primes up to N diverges as $\log \log N$.

- → **Time Complexity:** $\mathcal{O}(N \log \log N)$
- → **Space Complexity:** $\mathcal{O}(N)$

For practical CP limits ($N \approx 10^7$), $\mathcal{O}(N \log \log N)$ is extremely fast and often behaves almost like $\mathcal{O}(N)$.

The Inefficiency of the Standard Sieve

If the Sieve of Eratosthenes is so fast, why do we need anything else?

The Redundancy: Many composite numbers are crossed out multiple times.

- $\rightarrow$ 12 is crossed out when $i = 2$ $(2 \cdot 6)$.
- $\rightarrow$ 12 is crossed out again when $i = 3$ $(3 \cdot 4)$.

To achieve strict $\mathcal{O}(N)$ time, we must ensure every composite number is visited and crossed out **exactly once**.

The Linear Sieve: Mathematical Foundation

To visit every composite number C exactly once, we must represent it uniquely.

Unique Representation: Every integer $C > 1$ can be uniquely factored as:

$$C = p \cdot i$$

Where p is the **smallest prime factor (SPF)** of C, and $i \geq p$.

If we can guarantee that we only generate C when iterating over i and multiplying it by its smallest prime factor p, we achieve linear time.

The Linear Sieve: The Algorithm

1. Iterate i from 2 to N.
2. If i has not been marked, add it to our list of primes.
3. Iterate through our discovered primes p_j.
4. Mark $i \cdot p_j$ as composite.
5. **The Crucial Break Condition:** Stop the inner loop if $i \bmod p_j == 0$.

Why does the break condition work? If $i \bmod p_j == 0$, then p_j is the smallest prime factor of i. For any subsequent prime $p_k > p_j$, the composite number $C = i \cdot p_k$ will have p_j as its smallest prime factor, not p_k. We must leave it to be crossed out when the outer loop eventually reaches $i' = C/p_j$.

The Linear Sieve: Implementation

Pseudocode:

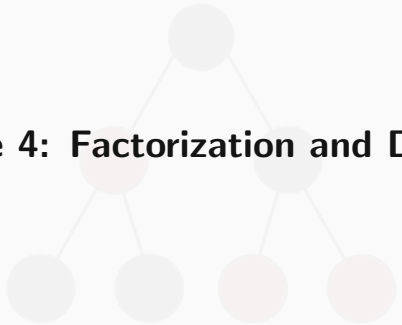
```
function linear_sieve(N):
    is_prime = boolean array initialized to true
    primes = empty list
    for i from 2 to N:
        if is_prime[i]:
            primes.append(i)
        for p in primes:
            if i * p > N: break
            is_prime[i * p] = false
            if i mod p == 0:
                break  // p is the SPF of i
    return primes
```

Linear vs. Standard Sieve

When to use which?

- → **Standard Sieve:** Easier to memorize, smaller constant factor. Often faster in practice for $N \leq 10^7$ due to simple array access patterns and CPU cache friendliness.
- → **Linear Sieve:** Strictly $\mathcal{O}(N)$. Extremely powerful because the same $\mathcal{O}(N)$ loop can be used to precompute multiplicative functions (like Euler's Totient function ϕ, or divisor counts) alongside the primes.

Pro-Tip: In CP, the array `is_prime` in the linear sieve is often replaced with an array `spf` (Smallest Prime Factor) to allow for $\mathcal{O}(\log K)$ prime factorization of multiple queries later.



Module 4: Factorization and Divisor Functions

Primality Testing: Trial Division

The Problem: Given a single integer N, determine if it is prime.

Mathematical Property: If N is a composite number, it can be factored into two integers, $N = a \cdot b$, where $a, b > 1$.

If both a and b were strictly greater than $\sqrt{N}$, then $a \cdot b > \sqrt{N} \cdot \sqrt{N} = N$, which is a contradiction. Therefore, at least one factor must be less than or equal to $\sqrt{N}$.

Conclusion: We only need to test for divisors up to $\lfloor \sqrt{N} \rfloor$. If no divisors are found in this range, N is prime.

Primality Testing: Implementation

Pseudocode:

```
function is_prime(N):
    if N < 2: return false
    for i from 2 to floor(sqrt(N)):
        if N mod i == 0:
            return false
    return true
```

Complexity Analysis:

- → **Time:** $\mathcal{O}(\sqrt{N})$
- → **Space:** $\mathcal{O}(1)$
- → *Note:* To avoid precision issues with floating-point 'sqrt()', it is safer to write the loop condition as 'i * i ¡= N' (using 64-bit integers to prevent overflow).

Integer Factorization: Trial Division

We can extend the trial division logic to find the prime factorization of N.

The Process:

1. Iterate i from 2 to $\sqrt{N}$.
2. If i divides N, divide N by i as many times as possible, recording i as a prime factor.
3. **Crucial Step:** After the loop, if the remaining value of N is greater than 1, it is a prime number itself and must be added to the list of factors.

Factorization: Implementation

Pseudocode:

```
function prime_factors(N):
    factors = empty list
    for i from 2 to floor(sqrt(N)):  // i * i <= N
        while N mod i == 0:
            factors.append(i)
            N = floor(N / i)
    if N > 1:
        factors.append(N)
    return factors
```

Time Complexity: $\mathcal{O}(\sqrt{N})$ worst-case (when N is prime).

Fast Factorization: The $\mathcal{O}(\log N)$ Trick

The Constraint Wall: What if you need to factor 10^5 different numbers, each up to 10^7? Calling an $\mathcal{O}(\sqrt{N})$ function 10^5 times will cause a Time Limit Exceeded (TLE) error.

The Solution: Recall the Sieve of Eratosthenes (or Linear Sieve). Instead of just marking 'true'/'false', we can store the **Smallest Prime Factor (SPF)** for each composite number during the sieve computation.

Fast Factorization: Implementation

Assuming we have precomputed an array 'spf' using a Sieve in $\mathcal{O}(MAX\_N \log \log MAX\_N)$ or $\mathcal{O}(MAX\_N)$:

Pseudocode (Answering a query):

```
function fast_factorization(N, spf):
    factors = empty list
    while N != 1:
        factors.append(spf[N])
        N = floor(N / spf[N])
    return factors
```

Complexity: Time $\mathcal{O}(\log N)$ per query, since N is divided by at least 2 at each step.

Divisor Functions: Introduction

Once we have the prime factorization of N, we can easily answer queries about its divisors without actually finding them all.

Let the unique prime factorization of N be:

$$N = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$$

Where p_i are distinct prime factors, and a_i are their respective exponents.

We will look at two fundamental multiplicative functions:

- $\tau(N)$ or $d(N)$: The total **number** of divisors.
- $\sigma(N)$: The **sum** of all divisors.

Number of Divisors: $\tau(N)$

Combinatorial Proof: Any divisor D of N must be formed by selecting a subset of N's prime factors.

For a specific prime factor p_i (which appears a_i times in N), the divisor D can contain p_i exactly $0, 1, 2, \ldots, a_i$ times. This gives exactly $(a_i + 1)$ independent choices for each prime factor.

Formula:

$$\tau(N) = (a_1 + 1) \cdot (a_2 + 1) \cdots (a_k + 1) = \prod_{i=1}^{k} (a_i + 1)$$

Example: $12 = 2^2 \cdot 3^1$. $\tau(12) = (2 + 1) \cdot (1 + 1) = 3 \cdot 2 = 6$. (The divisors are 1, 2, 3, 4, 6, 12).

Sum of Divisors: $\sigma(N)$

Algebraic Expansion: If we want the sum of all divisors, we can represent it as the product of the sums of all possible powers of each prime factor.

$$\sigma(N) = (1 + p_1 + p_1^2 + \cdots + p_1^{a_1}) \cdots (1 + p_k + p_k^2 + \cdots + p_k^{a_k})$$

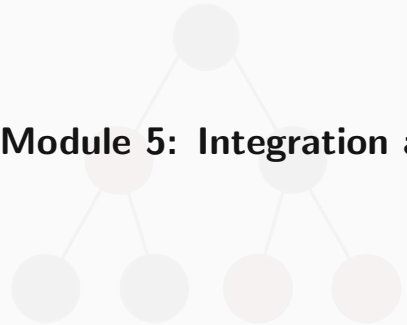
Each term in parenthesis is a geometric series. We can simplify this using the geometric series sum formula:

Formula:

$$\sigma(N) = \prod_{i=1}^{k} \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

Pro-Tips & CP "Gotchas"

- → **Multiplicative Property:** Both $\tau(N)$ and $\sigma(N)$ are multiplicative. This means if $\gcd(A, B) = 1$, then $\tau(A \cdot B) = \tau(A) \cdot \tau(B)$. This allows for $\mathcal{O}(N)$ precomputations alongside the Linear Sieve.
- → **Overflows with $\sigma(N)$:** The sum of divisors can easily exceed 32-bit integer limits even for relatively small inputs. Always use 64-bit integers.
- → **Modulo operations on $\sigma(N)$:** If a problem asks for $\sigma(N) \pmod M$, calculating the fraction $\frac{p_i^{a_i+1}-1}{p_i-1} \pmod M$ requires finding the Modular Inverse of $(p_i - 1)$ modulo M (refer back to Module 1).



Module 5: Integration and Q&A

Recognizing the Right Tool: Constraints

In Competitive Programming, the constraints dictate the algorithm. Let N be the primary input variable.

Constraint Guidelines:

- $N \leq 10^7$: $\mathcal{O}(N)$ operations are safe. Use the **Linear Sieve** or **Standard Sieve** for primes, factorization queries, or precomputing divisor functions.
- $N \leq 10^{12}$: $\mathcal{O}(\sqrt{N})$ is required. Use **Trial Division** for primality testing or factorization of a single number.
- $N \leq 10^{18}$: $\mathcal{O}(\log N)$ or $\mathcal{O}(1)$ is required. Rely on **Binary Exponentiation**, **Matrix Exponentiation**, or the **Euclidean Algorithm**.

Complexity Cheat Sheet

Algorithm	**Time Complexity**	**Space Complexity**
Euclidean Algorithm (GCD)	$\mathcal{O}(\log(\min(A, B)))$	$\mathcal{O}(1)$
Extended Euclidean	$\mathcal{O}(\log(\min(A, B)))$	$\mathcal{O}(1)$
Binary Exponentiation	$\mathcal{O}(\log B)$	$\mathcal{O}(1)$
Matrix Exponentiation	$\mathcal{O}(K^3 \log N)$	$\mathcal{O}(K^2)$
Sieve of Eratosthenes	$\mathcal{O}(N \log \log N)$	$\mathcal{O}(N)$
Linear Sieve	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Trial Division (Factorization)	$\mathcal{O}(\sqrt{N})$	$\mathcal{O}(\log N)$
Fast Factorization (with Sieve)	$\mathcal{O}(\log N)$ per query	$\mathcal{O}(N)$ precomp

Common CP Scenarios & Triggers

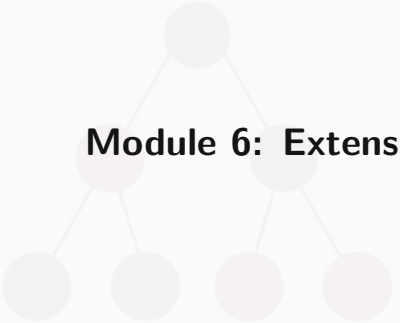
When reading a problem statement, look for these mathematical triggers:

- → **"Calculate $X/Y \pmod M$"** → Compute the Modular Inverse of Y modulo M. Use Fermat's Little Theorem if M is prime, else Extended Euclidean.
- → **"Find the N-th state of a linear process"** → Define the transition matrix and apply Matrix Exponentiation.
- → **"Output the answer modulo $10^9 + 7$"** → Modulo arithmetic is required at every addition, subtraction, and multiplication step.
- → **"Given 10^5 numbers, count their divisors"** → Precompute the Smallest Prime Factor (SPF) using a Linear Sieve, then use the $\tau(N)$ formula for each query.

Final Checklist Before Submission

Before submitting your code, verify these common failure points in number theory problems:

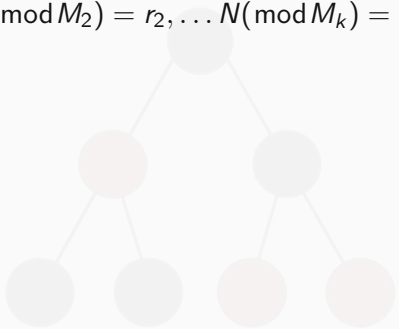
1. **Integer Overflow:** Are you using 64-bit integers (`long long` in C++) for intermediate multiplications before applying the modulo?
2. **Negative Modulo:** Does your code handle subtraction safely? Use `(A - B % M + M) % M`.
3. **Base Cases:** Does your logic handle $N = 0$ or $N = 1$ correctly, especially for prime sieves and exponentiation?
4. **Coprime Requirements:** If taking a modular inverse via Extended Euclidean, did you verify that $\gcd(A, M) = 1$?



Module 6: Extensions

Extensions: Chinese Remainder Theorem

Can we always construct a solution to some set of congruences?
$N(\mathrm{mod}\, M_1) = r_1, N(\mathrm{mod}\, M_2) = r_2, \ldots N(\mathrm{mod}\, M_k) = r_k$



Extensions: Chinese Remainder Theorem

Can we always construct a solution to some set of congruences?
$N(\mathrm{mod}\, M_1) = r_1, N(\mathrm{mod}\, M_2) = r_2, \ldots N(\mathrm{mod}\, M_k) = r_k$

Yes, provided $(M_i, M_j) = 1\ \forall i < j$. This is a constructive proof, providing a solution modulo $(\mathrm{mod}\, M := M_1 M_2 \ldots M_k)$. Recall the extended Euclidean algorithm, where terms can vanish when we use modular arithmetic.

We want to do something similar, where we sum k terms together where for each M_k all terms but one vanish.

- $\rightarrow$ compute $M = M_1 M_2 \ldots M_k$
- $\rightarrow$ For each $i = 1, 2, \ldots k$ compute $b_i = \frac{M}{M_i}$ which screens out most terms
- $\rightarrow$ For each $i = 1, 2, \ldots k$ compute $c_i \equiv b_i^{-1} (\mathrm{mod}\, m_i)$ guaranteed to exist since all m_i coprime
- $\rightarrow$ $N = a_1 b_1 c_1 + a_2 b_2 c_2 + \ldots a_k b_k c_k$ is your solution

Extension: Probabilistic Primality Testing (Miller-Rabin)

The Problem: Trial division is $\mathcal{O}(\sqrt{N})$ — too slow for $N \leq 10^{18}$.

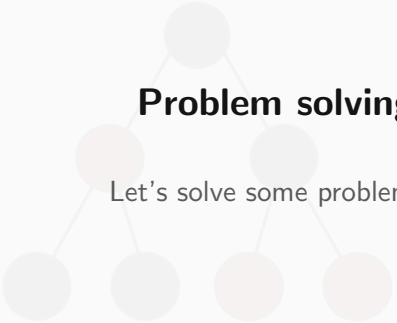
The Algorithm: To test if n is prime, write $n - 1 = 2^s \cdot d$ with d odd. For a randomly chosen base a:

1. Compute $x = a^d \bmod n$ via binary exponentiation.
2. If $x \equiv 1$ or $x \equiv -1 \pmod{n}$, this base *passes* — move to the next.
3. Repeatedly square x up to $s - 1$ times. If any square gives $x \equiv -1$, this base passes.
4. If none of the above triggered, n is **definitely composite**. Stop.
5. If all k bases pass, n is **probably prime**.

Extension: Probabilistic Primality Testing

In practice:

- → **Randomized:** k random bases → false positive probability $\leq 4^{-k}$.
- → **Deterministic:** Bases $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$ are correct for all $N < 3.3 \times 10^{24}$.
- → **Complexity:** $\mathcal{O}(k \log^2 N)$ per query.



Problem solving

Let's solve some problems!.